

SKILL: WEEK-4;

TASK-1:

```
harshitha18@LAPTOP-6E22AR6C:~$ nano task1.c
```

```
harshitha18@LAPTOP-6E22AR  X  +  v  task1.c *
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

void tokenize(char *input, char *tokens[], int *count)
{
    char *token;
    *count = 0;

    token = strtok(input, " \\t\\n");
    while (token != NULL && *count < MAX_TOKENS - 1)
    {
        tokens[*count] = token;
        (*count)++;
        token = strtok(NULL, " \\t\\n");
    }
    tokens[*count] = NULL;
}

int validate_tokens(char *tokens[], int count)
{
    if (count == 0)
        return 0;

    for (int i = 0; i < count; i++)
    {
        if (strlen(tokens[i]) == 0)
            return 0;
    }
    return 1;
}

int main()
{
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int count;

    while (1)
    {
        printf("Enter command: ");
        fflush(stdout);
        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            printf("\nExiting shell...\n");
            break;
        }

        if (strlen(input) == 1 && input[0] == '\\n')
        {
            printf("Empty command. Please enter a command.\n");
            continue;
        }

        tokenize(input, tokens, &count);

        if (!validate_tokens(tokens, count))
        {
            printf("Invalid command.\n");
            continue;
        }

        if (strcmp(tokens[0], "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        printf("Tokens:\n");
        for (int i = 0; i < count; i++)
        {
            printf("Token %d: %s\n", i + 1, tokens[i]);
        }

        pid_t pid = fork();
        if (pid < 0)
        {
            perror("fork failed");
            continue;
        }
        if (pid == 0)
        {
            execvp(tokens[0], tokens);
            perror("execvp failed");
            exit(EXIT_FAILURE);
        }
        else
        {
            waitpid(pid, NULL, 0);
        }
    }
    return 0;
}
```

```
harshitha18@LAPTOP-6E22AR  X  +  v  task1.c *
GNU nano 8.7.1
printf("\nExiting shell...\n");
break;
}

if (strlen(input) == 1 && input[0] == '\\n')
{
    printf("Empty command. Please enter a command.\n");
    continue;
}

tokenize(input, tokens, &count);

if (!validate_tokens(tokens, count))
{
    printf("Invalid command.\n");
    continue;
}

if (strcmp(tokens[0], "exit") == 0)
{
    printf("Exiting shell...\n");
    break;
}

printf("Tokens:\n");
for (int i = 0; i < count; i++)
{
    printf("Token %d: %s\n", i + 1, tokens[i]);
}

pid_t pid = fork();
if (pid < 0)
{
    perror("fork failed");
    continue;
}
if (pid == 0)
{
    execvp(tokens[0], tokens);
    perror("execvp failed");
    exit(EXIT_FAILURE);
}
else
{
    waitpid(pid, NULL, 0);
}
}
return 0;
}
```

```
harshitha18@LAPTOP-6E22AR:~$ nano task1.c
harshitha18@LAPTOP-6E22AR:~$ ls
OSSP      file1.txt  fork_example.c  process.c  task4.c
backup.txt file1.txt  hello           task1.c    task5.c
exec_example file2.txt  hello.c         task1.c    task5.c
exec_example.c fork_example lexer_parser.c.save task4
harshitha18@LAPTOP-6E22AR:~$ gcc task1.c -o task1
harshitha18@LAPTOP-6E22AR:~$ ls
OSSP      file1.txt  fork_example.c  process.c  task4.c
backup.txt file1.txt  hello           task1.c    task5.c
exec_example file2.txt  hello.c         task1.c    task5.c
exec_example.c fork_example lexer_parser.c.save task4
harshitha18@LAPTOP-6E22AR:~$ ./task1
Enter command: ls
Tokens:
Token 1: ls
OSSP      file1.txt  fork_example.c  process.c  task4.c
backup.txt file1.txt  hello           task1.c    task5.c
exec_example file2.txt  hello.c         task1.c    task5.c
exec_example.c fork_example lexer_parser.c.save task4
Enter command: pwd
Tokens:
Token 1: pwd
/home/harshitha18
Enter command: echo Hello World
Tokens:
Token 1: echo
Token 2: Hello
Token 3: World
Hello World
Enter command: ls
Tokens:
Token 1: ls
OSSP      file1.txt  fork_example.c  process.c  task4.c
backup.txt file1.txt  hello           task1.c    task5.c
exec_example file2.txt  hello.c         task1.c    task5.c
exec_example.c fork_example lexer_parser.c.save task4
```

```
harshitha18@LAPTOP-6E22AR:~$ ls -l
Enter command: ls -l
Tokens:
Token 1: ls
Token 2: -l
total 172
drwxr-xr-x 11 harshitha18 harshitha18 4096 Aug 14 05:17 OSSP
-rw-r--r-- 1 harshitha18 harshitha18 14 Aug 7 14:38 backup.txt
-rwxr-xr-x 1 harshitha18 harshitha18 16000 Aug 7 13:52 exec_example
-rw-r--r-- 1 harshitha18 harshitha18 170 Aug 7 13:52 exec_example.c
-rw-r--r-- 1 harshitha18 harshitha18 0 Aug 7 14:36 file1.txt
-rw-r--r-- 1 harshitha18 harshitha18 14 Aug 7 14:37 file1.txt
-rw-r--r-- 1 harshitha18 harshitha18 14 Aug 7 14:38 file2.txt
-rwxr-xr-x 1 harshitha18 harshitha18 16088 Aug 7 13:51 fork_example
-rw-r--r-- 1 harshitha18 harshitha18 443 Aug 7 13:51 fork_example.c
-rwxr-xr-x 1 harshitha18 harshitha18 15952 Aug 7 16:21 hello
-rw-r--r-- 1 harshitha18 harshitha18 71 Aug 7 16:20 hello.c
-rw-r--r-- 1 harshitha18 harshitha18 12313 Aug 14 05:10 lexer_parser.c.save
-rw-r--r-- 1 harshitha18 harshitha18 1274 Jul 29 07:07 process.c
-rwxr-xr-x 1 harshitha18 harshitha18 16632 Aug 16 09:47 task1
-rw-r--r-- 1 harshitha18 harshitha18 1998 Aug 16 09:46 task1.c
-rwxr-xr-x 1 harshitha18 harshitha18 16520 Aug 7 15:22 task4
-rw-r--r-- 1 harshitha18 harshitha18 4484 Aug 7 15:21 task4.c
-rwxr-xr-x 1 harshitha18 harshitha18 16464 Aug 7 15:33 task5
-rw-r--r-- 1 harshitha18 harshitha18 3156 Aug 7 15:30 task5.c
Enter command:
Empty command. Please enter a command.
Enter command: hello 123
Tokens:
Token 1: hello
Token 2: 123
execvp failed: No such file or directory
Enter command: echo this is Task 1
Tokens:
Token 1: echo
Token 2: this
Token 3: is
Token 4: Task
```

```

Tokens:
Token 1: echo
Token 2: this
Token 3: is
Token 4: Task
Token 5: 1
this is Task 1
Enter command: date
Tokens:
Token 1: date
Sun Aug 16 09:48:37 UTC 2026
Enter command: whoami
Tokens:
Token 1: whoami
harshitha18
Enter command: exit
Exiting shell...
harshitha18@LAPTOP-6E22AR6C:~$ |

```

successfully demonstrates the core mechanics of a custom shell, focusing on parsing raw user input into structured tokens and validating them. By leveraging foundational system calls like `fork()`, `execvp()`, and `waitpid()`, it establishes a robust workflow for isolated command execution and orderly process management.

TASK-2:

```
harshitha18@LAPTOP-6E22AR6C:~$ nano task2.c
```

```

harshitha18@LAPTOP-6E22AR X + v task2.c
GNU nano 3.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 100
#define MAX_LEN 1024

int parse_command(char *input, char *args[]) {
    int argc = 0;
    int in_quote = 0;
    char quote = '\0';
    char *p = input;

    while (*p) {
        while (*p == ' ' || *p == '\t' || *p == '\n')
            p++;

        if (*p == '\0')
            break;

        args[argc++] = p;

        while (*p) {
            if (in_quote && (*p == '\'' || *p == '"')) {
                in_quote = 1;
                quote = *p;
                memmove(p, p + 1, strlen(p));
                continue;
            }

            if (in_quote && *p == quote) {
                in_quote = 0;
                memmove(p, p + 1, strlen(p));
                continue;
            }

            if (!in_quote && (*p == ' ' || *p == '\t' || *p == '\n')) {
                *p = '\0';
                p++;
                break;
            }
        }
        p++;
    }
}

```

Help Write Out Where Is Cut Execute Location Undo Set Mark To Bracket Previous Back
Exit Read File Replace Paste Justify Go To Line Redo Copy Where Was Next Forward

```
harshitha18@LAPTOP-5E22A8  ×  +  ▾
GNU nano 8.7.1 task2.c
    if (!in_quote) {
        return -1;
    }

    args[argc] = NULL;
    return argc;
}

int main() {
    char input[MAX_LEN];
    char *args[MAX_ARGS];

    while (1) {
        printf("myShell>> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        if (strcmp(input, "exit\n") == 0)
            break;

        int argc = parse_command(input, args);

        if (argc == -1) {
            printf("Syntax Error: Unmatched single quote.\n");
            continue;
        }

        if (argc == 0)
            continue;

        printf("\nParsing Result:\n");
        for (int i = 0; i < argc; i++)
            printf("Argument %d: %s\n", i + 1, args[i]);

        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
            continue;
        }

        if (pid == 0) {
            execvp(args[0], args);

```

```

        if (pid == 0) {
            execvp(args[0], args);
            perror("execvp");
            exit(1);
        } else {
            printf("\nChild PID: %d\n", pid);

            int status;
            waitpid(pid, &status, 0);

            printf("\nCommand execution completed.\n\n");
        }
    }

    return 0;
}

```

```
harshitha18@LAPTOP-6E22AR X + v
harshitha18@LAPTOP-6E22AR6C:~$ ./task2
myShell>> echo Hello

Parsing Result:
Argument 1: echo
Argument 2: Hello
Child PID: 1175
Hello
Command execution completed.

myShell>> echo 'Hello World'

Parsing Result:
Argument 1: echo
Argument 2: Hello World
Child PID: 1178
Hello World
Command execution completed.

myShell>> echo 'Hello World
Syntax Error: Unmatched single quote.
myShell>> echo '$HOME'

Parsing Result:
Argument 1: echo
Argument 2: $HOME
Child PID: 1183
$HOME
Command execution completed.

myShell>> echo 'Hello    World'

Parsing Result:
Argument 1: echo
Argument 2: Hello    World
Child PID: 1184
Hello    World
Command execution completed.
```

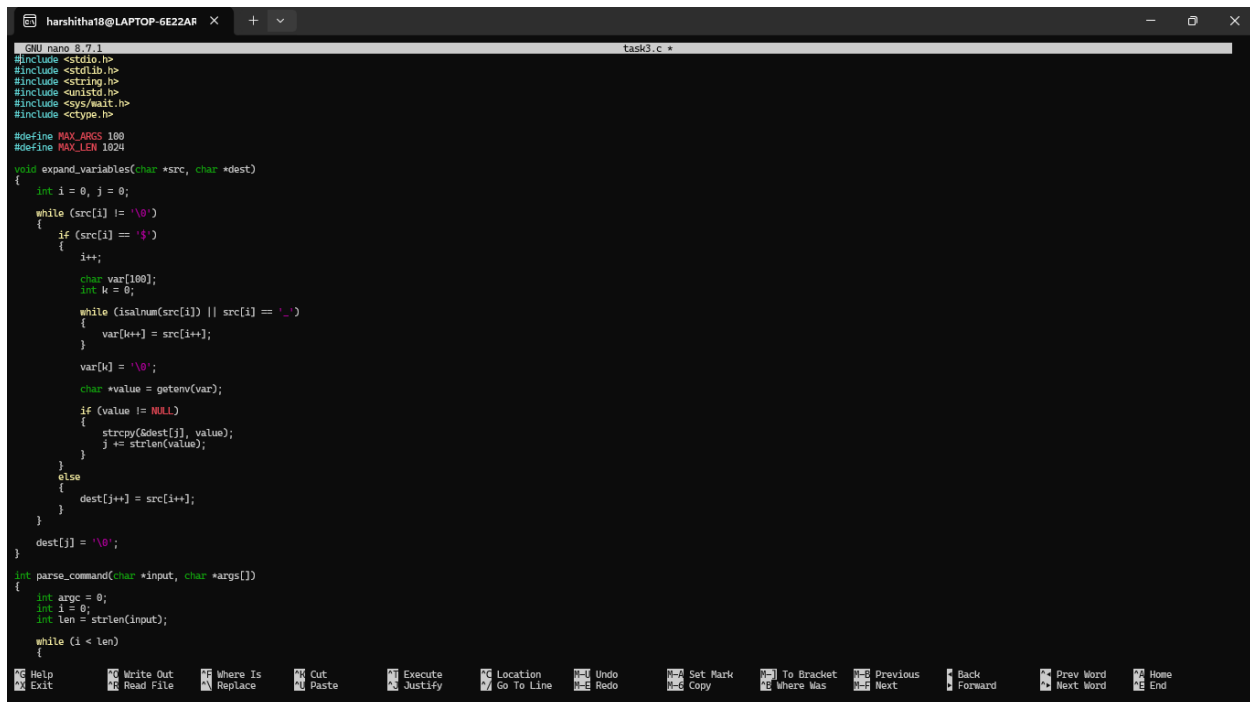
```
myShell>> exit
harshitha18@LAPTOP-6E22AR6C:~$ |
```

successfully implemented single-quote handling, argument parsing, literal content preservation, and syntax-error detection.

It also demonstrated process creation using `fork()`, command execution using `execvp()`, and synchronization using `waitpid()`.

TASK-3:

```
harshitha18@LAPTOP-6E22AR6C:~$ nano task3.c
```



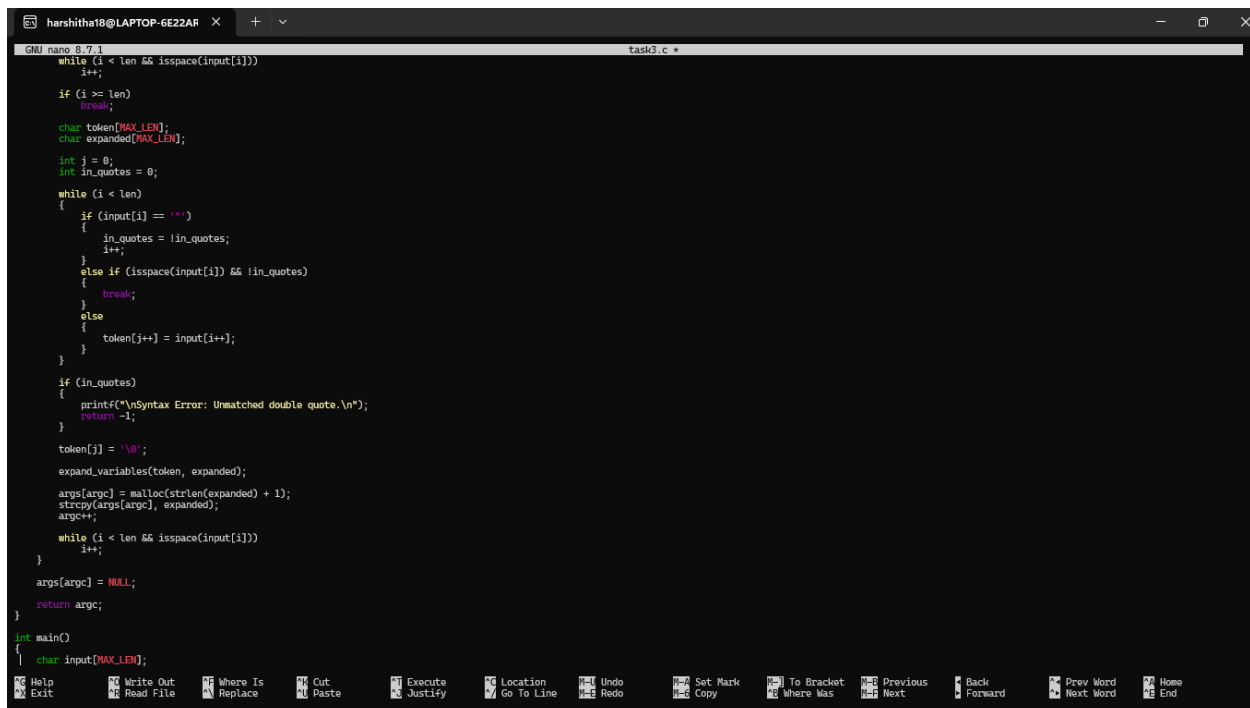
```
GNU nano 8.7.1 task3.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/malloc.h>
#include <ctype.h>

#define MAX_ARGS 100
#define MAX_LEN 1024

void expand_variables(char *src, char *dest)
{
    int i = 0, j = 0;
    while (src[i] != '\0')
    {
        if (src[i] == '$')
        {
            i++;
            char var[100];
            int k = 0;
            while (isalnum(src[i]) || src[i] == '_')
            {
                var[k++] = src[i++];
            }
            var[k] = '\0';
            char *value = getenv(var);
            if (value != NULL)
            {
                strcpy(&dest[j], value);
                j += strlen(value);
            }
            else
            {
                dest[j++] = src[i++];
            }
        }
        else
        {
            dest[j++] = src[i++];
        }
    }
    dest[j] = '\0';
}

int parse_command(char *input, char *args[])
{
    int argc = 0;
    int i = 0;
    int len = strlen(input);
    while (i < len)
    {

```



```

        while (i < len && isspace(input[i]))
            i++;
        if (i >= len)
            break;
        char token[MAX_LEN];
        char expanded[MAX_LEN];
        int j = 0;
        int in_quotes = 0;
        while (i < len)
        {
            if (input[i] == '"')
            {
                in_quotes = !in_quotes;
                i++;
            }
            else if (isspace(input[i]) && !in_quotes)
            {
                break;
            }
            else
            {
                token[j++] = input[i++];
            }
        }
        if (in_quotes)
        {
            printf("\nSyntax Error: Unmatched double quote.\n");
            return -1;
        }
        token[j] = '\0';
        expand_variables(token, expanded);
        args[argc] = malloc(strlen(expanded) + 1);
        strcpy(args[argc], expanded);
        argc++;
        while (i < len && isspace(input[i]))
            i++;
    }
    args[argc] = NULL;
    return argc;
}

int main()
{
    char input[MAX_LEN];

```

```
harshitha18@LAPTOP-6E22AR  X  +  v  GNU nano 8.7.1  task3.c *  
  
while (1)  
{  
    printf("\nmyShell> ");  
    fflush(stdout);  
    if (fgets(input, sizeof(input), stdin) == NULL)  
        break;  
    input[strcspn(input, "\n")] = '\0';  
    if (strcmp(input, "exit") == 0)  
        break;  
    if (strlen(input) == 0)  
        continue;  
    char *args[MAX_ARGS];  
    int argc = parse_command(input, args);  
    if (argc == -1)  
        continue;  
    printf("\n--- Parsed Arguments ---\n");  
    for (int i = 0; i < argc; i++)  
    {  
        printf("Argument %d: %s\n", i + 1, args[i]);  
    }  
    printf("\nChild Process\n");  
    pid_t pid = fork();  
    if (pid < 0)  
    {  
        perror("fork");  
    }  
    else if (pid == 0)  
    {  
        execvp(args[0], args);  
        perror("execvp");  
        exit(1);  
    }  
    else  
    {  
        printf("PID = %d\n\n", pid);  
        wait(NULL);  
        printf("Child process completed.\n");  
    }  
}
```

```
    printf("PID = %d\n\n", pid);  
    wait(NULL);  
    printf("Child process completed.\n");  
}  
  
for (int i = 0; i < argc; i++)  
    free(args[i]);  
}  
  
return 0;  
}  
}
```

```
harshitha18@LAPTOP-6E22AR X + v
harshitha18@LAPTOP-6E22AR6C:~$ gcc task3.c -o task3
harshitha18@LAPTOP-6E22AR6C:~$ ls
OSDP      exec_example.c  file2.txt  hello      process.c  task2     task3.c  task5
backup.txt file1.txt       fork_example hello.c     task1     task2.c  task4     task5.c
exec_example file1.txt       fork_example.c  lexer_parser.c.save task1.c    task3     task4.c
harshitha18@LAPTOP-6E22AR6C:~$ ./task3

myShell> echo "Hello $USER"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello harshitha18

Child Process
PID = 1293

Hello harshitha18
Child process completed.

myShell> echo "Hello World"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Child Process
PID = 1294

Hello World
Child process completed.

myShell> echo "User: $USER"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: User: harshitha18

Child Process
PID = 1295

User: harshitha18
Child process completed.

myShell> echo "Hello $USER from $HOME"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello harshitha18 from /home/harshitha18

Child Process
PID = 1298

Hello harshitha18 from /home/harshitha18
Child process completed.

myShell> echo "Hello World"

Syntax Error: Unmatched double quote.

myShell> ls

--- Parsed Arguments ---
Argument 1: ls

Child Process
PID = 1299

OSDP      exec_example.c  file2.txt  hello      process.c  task2     task3.c  task5
backup.txt file1.txt       fork_example hello.c     task1     task2.c  task4     task5.c
exec_example file1.txt       fork_example.c  lexer_parser.c.save task1.c    task3     task4.c
Child process completed.

myShell> pwd

--- Parsed Arguments ---
Argument 1: pwd

Child Process
PID = 1302

/home/harshitha18
Child process completed.

myShell> echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello

Child Process
PID = 1305

Hello
Child process completed.

myShell> exit
harshitha18@LAPTOP-6E22AR6C:~$ |
```

successfully implemented double-quote handling, space preservation, variable expansion, and nested token parsing in the shell.

It also validated unmatched quotes and executed valid commands using child processes with `fork ()` and `execvp ()`.

TASK-4:

```
harshitha18@LAPTOP-6E22AR6C:~$ ls
OSSP          file1.txt     fork_example.c  process.c     task2.c       task4.c
backup.txt    file1.txt     hello           task1         task3         task5
exec_example  file2.txt     hello.c         task1.c       task3.c       task5.c
exec_example.c fork_example  lexer_parser.c.save task2         task4
harshitha18@LAPTOP-6E22AR6C:~$ nano task4.c
```

```
harshitha18@LAPTOP-6E22AR6C:~$ nano task4.c
GNU nano 2.7.1 task4.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

int parse_command(char *input, char *args[])
{
    int argc = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        while (isspace(input[i]))
            i++;

        if (input[i] == '\0')
            break;

        char token[MAX_INPUT];
        int j = 0;

        while (input[i] != '\0' && !isspace(input[i]))
        {
            if (input[i] == '\\')
            {
                i++;
                if (input[i] != '\0')
                {
                    token[j++] = input[i++];
                }
            }
            else
            {
                token[j++] = input[i++];
            }
        }

        token[j] = '\0';

        args[argc] = malloc(strlen(token) + 1);
        strcpy(args[argc], token);
        argc++;
    }

    args[argc] = NULL;
    return argc;
}

int main()
{
    char input[MAX_INPUT];

    while (1)
    {
        printf("myShell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        char *args[MAX_ARGS];
        int argc = parse_command(input, args);

        printf("\nParent Process PID: %d\n", getpid());

        pid_t pid = fork();
        if (pid < 0)
        {
            perror("fork");
            return 1;
        }
        if (pid == 0)
        {
            printf("Created Child PID: %d\n", getpid());
            execvp(args[0], args);
        }
    }
}
```

```
harshitha18@LAPTOP-6E22AR6C:~$ nano task4.c
GNU nano 2.7.1 task4.c
}
    }

    args[argc] = NULL;
    return argc;
}

int main()
{
    char input[MAX_INPUT];

    while (1)
    {
        printf("myShell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        char *args[MAX_ARGS];
        int argc = parse_command(input, args);

        printf("\nParent Process PID: %d\n", getpid());

        pid_t pid = fork();
        if (pid < 0)
        {
            perror("fork");
            return 1;
        }
        if (pid == 0)
        {
            printf("Created Child PID: %d\n", getpid());
            execvp(args[0], args);
        }
    }
}
```

```
        perror(args[0]);
        exit(2);
    }
    else
    {
        printf("Child Process PID: %d\n", pid);

        int status;

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }
    }

    for (int i = 0; i < argc; i++)
        free(args[i]);
}

return 0;
}
|
```

Help **Write Out** **Where Is** **Cut** **Execute** **Location** **Undo**
Exit **Read File** **Replace** **Paste** **Justify** **Go To Line** **Redo**

```
harshitha18@LAPTOP-6E22AR X + v
harshitha18@LAPTOP-6E22AR6C:~$ gcc task4.c -o task4
harshitha18@LAPTOP-6E22AR6C:~$ ls
OSSP      file1.txt  fork_example.c  process.c  task2.c  task4.c
backup.txt file1.txt  hello           task1      task3    task5
exec_example file2.txt  hello.c         task1.c   task3.c  task5.c
exec_example.c fork_example lexer_parser.c.save task2      task4
harshitha18@LAPTOP-6E22AR6C:~$ gcc task4.c -o task4
harshitha18@LAPTOP-6E22AR6C:~$ ./task4
myShell$ ls

Parent Process PID: 1428
Child Process PID: 1429
Created Child PID: 1429
OSSP      file1.txt  fork_example.c  process.c  task2.c  task4.c
backup.txt file1.txt  hello           task1      task3    task5
exec_example file2.txt  hello.c         task1.c   task3.c  task5.c
exec_example.c fork_example lexer_parser.c.save task2      task4
Child exited with status: 0
myShell$ echo Hello\ World

Parent Process PID: 1428
Child Process PID: 1430
Created Child PID: 1430
Hello World
Child exited with status: 0
myShell$ echo Hello\!

Parent Process PID: 1428
Child Process PID: 1431
Created Child PID: 1431
Hello!
Child exited with status: 0
myShell$ echo Hello\!

Parent Process PID: 1428
Child Process PID: 1432
Created Child PID: 1432
Hello!
Child exited with status: 0
myShell$ echo \$USER

Parent Process PID: 1428
Child Process PID: 1433
Created Child PID: 1433
$USER
Child exited with status: 0
myShell$ echo This\ is\ a\ test

Parent Process PID: 1428
Child Process PID: 1434
Created Child PID: 1434
```

```
Parent Process PID: 1428
Child Process PID: 1434
Created Child PID: 1434
This is a test
Child exited with status: 0
myShell$ echo This\ is\ a\ test

Parent Process PID: 1428
Child Process PID: 1435
Created Child PID: 1435
This is a test
Child exited with status: 0
myShell$ ls my\ file.txt

Parent Process PID: 1428
Child Process PID: 1438
Created Child PID: 1438
ls: cannot access 'my file.txt': No such file or directory
Child exited with status: 2
myShell$ ls @

Parent Process PID: 1428
Child Process PID: 1439
Created Child PID: 1439
ls: cannot access '@': No such file or directory
Child exited with status: 2
myShell$ echo Hello World Test

Parent Process PID: 1428
Child Process PID: 1442
Created Child PID: 1442
Hello World Test
Child exited with status: 0
myShell$ exit
harshitha18@LAPTOP-6E22AR6C:~$ |
```

successfully implemented escape sequences, escaped spaces, and special character handling while preserving the correct command arguments.

It also created child processes using `fork()`, executed commands using `execvp()`, and handled child process exit statuses and errors.